

TRABAJO FINAL DE PROGRAMACIÓN EL JUEGO

PRIMERA ENTREGA: ANÁLISIS Y DISEÑO PREVIO

VERÓNICA ACEVES

BERUKTAWIT CUETO

NEREA GONZÁLEZ

ÍNDICE

Introducción.....	3
1. Propuesta inicial del juego.....	4
1. Nombre provisional del juego.....	4
2. Descripción general.....	4
3. Objetivo del juego.....	4
4. Mecánica principal.....	4
5. Tipo de usuario al que va dirigido.....	5
6. Condición de victoria, derrota o finalización.....	5
7. Sistema de puntuación, vidas, turnos, niveles o elemento equivalente	5
8. Tecnologías previstas.....	6
9. Sistema de persistencia elegido.....	7
2. Funcionalidades previstas.....	8
3. Sistema de persistencia.....	11
4. Diseño de datos.....	12
5. Diseño inicial de la aplicación. Diagrama MVC.....	16
Capa Modelo.....	16
Diagrama de clases UML.....	16
Capa Vista.....	17
Diagrama de flujo.....	17
Vistas.....	18
Capa Controlador.....	20
Diagrama de Clases Controller.....	20
Capa Persistencia.....	21
Diagrama Completo MVC.....	21
Patrón M-V-C.....	22
Diagrama de Clases M-V-C.....	22

Introducción

Este documento constituye la primera y segunda entrega del trabajo final de la asignatura de programación y funciona como una memoria de trabajo del proyecto. En él se recoge el análisis previo y el diseño inicial de la aplicación, incluyendo todos los puntos exigidos por la guía.

Gracias a la división de roles dentro del equipo, cada integrante asume la responsabilidad principal de una parte específica del proyecto, lo que permite una organización estructurada y una alineación directa con la arquitectura MVC. Esta distribución facilita una dirección clara en cada una de las capas del sistema, favoreciendo el desarrollo ordenado y coherente de la aplicación.

No obstante, es necesario subrayar que la línea de trabajo del equipo se basa en la participación activa de todas las integrantes en el desarrollo global del proyecto, colaborando en todas las capas del sistema, compartiendo decisiones de diseño, programación y análisis, garantizando así un reparto equilibrado del esfuerzo y la implicación en cada una de las fases y áreas del proyecto.

El objetivo de esta memoria es servir como base de referencia para el desarrollo del videojuego, garantizando una estructura clara, coherente y escalable antes de iniciar la implementación.

1. Propuesta inicial del juego

1. Nombre provisional del juego

Clueda: Misterio en el Castillo

2. Descripción general

Aplicación de escritorio para un juego de mesa monojugador, de estilo narrativo cuyo objetivo es resolver una serie de asesinatos.

El usuario encarna a La Dama Beatrice, una humana con sangre nigromante que se dedica al oficio de investigadora forense, que es contratada debido a su capacidad para comunicarse con el fantasma del Castillo. A través de esta conexión, deberá reconstruir y desvelar la historia de los trágicos acontecimientos ocurridos en el Castillo de Vallenwood.

3. Objetivo del juego

Averiguar en cada nivel quién es el asesino, en qué habitación se ha cometido el crimen y qué arma se ha utilizado, para así revelar la crónica completa.

4. Mecánica principal

Al iniciar una partida, el jugador escoge un nivel o historia. Por rondas, podrá interactuar con los elementos de la pantalla para formular hipótesis o hacer directamente una acusación.

Podrá pedir pistas para resolver el caso, que le ayudarán a reconstruir el caso, limitadas según la dificultad del nivel elegido.

Al finalizar la partida, el jugador podrá consultar toda la Crónica de Muerte correspondiente al nivel.

5. Tipo de usuario al que va dirigido

El juego está dirigido a todos los públicos a partir de 8 años, con especial interés en jugadores que disfruten de experiencias de misterio, deducción e investigación narrativa.

6. Condición de victoria, derrota o finalización

La partida finaliza cuando el jugador realiza una acusación, se queda sin puntos o agota el número de rondas disponibles.

Cada partida tiene como resultado victoria o derrota, sin posibilidad de empate.

La victoria cuando la acusación es completamente correcta, mientras que la derrota se produce si uno o varios de los elementos (personaje, arma o escenario) son incorrectos o si se alcanza alguna condición de fin de partida sin haber resuelto el caso.

7. Sistema de puntuación, vidas, turnos, niveles o elemento equivalente

El jugador comienza la partida con una cantidad de puntos iniciales determinada por el nivel de dificultad seleccionado.

Durante el desarrollo del juego, investigará el caso de dos maneras; interactuando con los elementos de la interfaz para obtener información, o solicitando una pista narrativa por turno, la cual ofrece información de forma indirecta y conlleva una penalización de 10 puntos.

En cada ronda, el usuario podrá realizar un interrogatorio formulando una hipótesis mediante la selección de un personaje, un arma y un escenario. En caso de que la hipótesis sea incorrecta, se restarán 10 puntos.

El jugador también tendrá la opción de realizar una acusación directamente, seleccionando los tres elementos, lo que finalizará la partida con resultado de victoria o derrota.

Los niveles tienen asociados una dificultad:

- a. Fácil: Niveles 1-3
 - i. Número de puntos iniciales: 100
 - ii. Número de rondas: 12
 - iii. Número de pistas: 3
- b. Difícil: Niveles 4-5
 - i. Número de puntos iniciales: 70
 - ii. Número de rondas: 8
 - iii. Número de pistas: 1

8. Tecnologías previstas

- a. Java, lenguaje principal de programación para implementar la lógica del juego y la gestión de datos.
- b. MVC (Modelo-Vista-Controlador), patrón de arquitectura utilizado para organizar el código y separar responsabilidades.
- c. JavaFX, framework para el desarrollo de la interfaz gráfica de usuario.
- d. FXML, lenguaje para la definición de las distintas vistas de la aplicación, separando la interfaz de la lógica.
- e. CSS, lenguaje para la personalización del diseño visual de la aplicación.
- f. Scene Builder, herramienta de diseño visual para la creación de interfaces FXML.
- g. Gradle, herramienta de automatización y gestión del proyecto (compilación, dependencias y ejecución).
- h. GitHub, sistema de control de versiones para el seguimiento del desarrollo.
- i. Eclipse IDE, entorno de desarrollo integrado para la programación y depuración del proyecto.

9. Sistema de persistencia elegido

Tanto los datos necesarios para el desarrollo del juego como el historial de partidas se guardarán en una base de datos documental en la nube.

Mediante un sistema CRUD, implementado a través de clases DAO, utilizaremos la plataforma de MongoDB Atlas como sistema de persistencia, cuya interfaz gráfica aligera los tiempos de trabajo.

Necesitaremos un clúster en la nube para el almacenamiento de los datos del juego con documentos JSON.

2. Funcionalidades previstas

Tabla General de Funcionalidades, tanto obligatorias como opcionales, previstas en el diseño del programa, incluyendo funciones de modelo (lógica y persistencia), controlador y vista.

COD	FUNCIONALIDAD	DESCRIPCIÓN	TIPO
F01	Iniciar partida	Permite iniciar una nueva partida creando una instancia de Partida y asociándole un Jugador.	Obligatoria
F02	Cargar partida	Permite recuperar una partida previamente guardada desde la base de datos, restaurando su estado completo para continuar el juego.	Obligatoria
F03	Configurar partida	Permite seleccionar el nivel (caso), que define, según su dificultad, puntos iniciales, rondas y pistas.	Obligatoria
F04	Gestión del flujo de partida	Coordina el desarrollo de la partida en curso, gestionando la interacción entre vista y modelo y asegurando la actualización del estado tras cada acción del jugador.	Obligatoria

F05	Guardar partida y salir	Permite almacenar el estado actual de una partida en curso en la base de datos para continuar posteriormente y regresar al menú principal de la aplicación.	Obligatoria
F06	Realizar hipótesis	El jugador selecciona personaje, arma y escenario. Se valida la información contra el Nivel y aplica penalización si es incorrecta.	Obligatoria
F07	Solicitar pista	Permite obtener una pista del Nivel, reduciendo las pistas disponibles y restando puntos al jugador.	Obligatoria
F08	Realizar acusación	Permite resolver el caso comparando los tres elementos con la solución del Nivel. Finaliza la partida con resultado de VICTORIA o DERROTA.	Obligatoria
F09	Persistir historial de partidas	Almacena el resumen al final de la partida (jugador, nivel, puntos finales y resultado).	Obligatoria

F10	Finalizar partida	Al finalizar la partida, muestra el resultado (victoria/derrota), la historia completa del nivel y el historial de las últimas 10 partidas.	Obligatoria
F11	Feedback visual de hipótesis	Muestra aciertos y errores mediante código de colores: los elementos correctos se iluminan y los incorrectos se oscurecen.	Obligatoria
F12	Información contextual	Los elementos del juego (personajes, armas y escenarios) muestran información adicional mediante pop-ups al interactuar.	Obligatoria
F13	Música de fondo	Reproduce música de ambiente durante la partida.	Opcional
F14	Efectos de sonido	Incluye efectos sonoros para acciones del juego.	Opcional

3. Sistema de persistencia

Para el desarrollo de este proyecto nos hemos decantado por la opción C: Base de datos NoSQL (MongoDB).

La razón por la que hemos optado por una base de datos NoSQL es porque nuestra lógica de juego está basada en la programación orientada a objetos y MongoDB, al ser una base de datos documental: nos permite guardar los objetos con los que trabajamos con libertad directamente en formato BSON/JSON, evitará que tengamos que crear múltiples tablas y complejas sentencias JOIN de SQL, facilitará el mapeo de los datos a través de las clases DAO y nos proporciona una gran flexibilidad a la hora de hacer cambios sobre la marcha.

Interactuaremos con la base de datos en los siguientes casos:

- Registro de Jugadores: Al iniciar la aplicación o crear una nueva partida, se registrará o recuperará la información del objeto jugador.
- Cargar partida: permite recuperar una partida que haya sido guardada con anterioridad. No puede haber más de una partida cargada
- Configurar partida: una vez se haya escogido el nivel, se accederá a la historia de cada caso.
- Guardar partida y salir: durante el transcurso del juego al guardar la partida guardará la información del estado actual de la partida, lo cual nos permitirá ejecutar cargar partida.
- Historial y Finalización : Cuando el método finalizarPartida() se ejecuta, el estado cambia a finalizada y se guarda el resultado (victoria o derrota). La aplicación leerá después estos documentos finalizados para mostrar el historial del jugador.

4. Diseño de datos

1. Nombre de la base de datos: Cluedo_db
2. Colección o colecciones previstas: vamos a tener las siguientes colecciones:
 - a. Jugador: almacena el perfil de los usuarios (id, nombre).
 - b. Nivel: contiene mayor parte de la información del juego: la dificultad, los escenarios, armas, descripciones, asesino, pistas.
 - c. Partida: guarda el estado dinámico del juego, vinculando al jugador, y al nivel correspondiente y el progreso de las rondas.
 - d. Arma, escenario y personaje: contendrá información de cada uno de los elementos.
3. Estructura de documento JSON:

Colección Nivel

```
_id
6a2e997416956d3cdb89a749
idNivel
"NIVEL_02"
dificultad
"FACIL"
asesino
Object
escenarioCrimen
Object
armaCrimen
Object
descripcion
"A las 8 es la hora de cambio de turno de los caballeros, y fue a esa h..."
pistas
Array (3)
pistaActual
0
```

Colección Partida:

```
_id  
6a3012d1c7f4fd5fcbd7c1cf  
idPartida  
"PTD_001"  
jugador  
Object  
idJugador  
"J_001"  
nombre  
"Beruk"  
nivel  
Object  
rondaActual  
2  
puntosActuales  
60  
pistasRestantes  
0  
estado  
"EN_CURSO"  
resultado  
null  
ultimoAcertoPersonaje  
true  
ultimoAcertoArma  
false  
ultimoAcertoEscenario  
false
```

Colección Juego:

```
_id  
"e19b25a4-98e8-496b-899d-4e5879109987"  
nombre  
"vera"
```

Trabajo Final: Misterio en el Castillo

Colección Arma:

```
_id
6a301343c7f4fd5fcbd7c1dc
idArma
"A_CANDELABRO"
nombre
"Candelabro de oro élfico"
descripcion
"Candelabro de tres brazos forjado con el oro de los bosques. Su estruc..."
```

Colección Escenario:

```
_id
6a301375c7f4fd5fcbd7c1e4
idEscenario
"E_DORMITORIO"
nombre
"Dormitorio Principal"
descripcion
"Estancia de techos altos y paredes cubiertas de tapices heráldicos. Lo..."
```

Colección Personaje:

```
_id
6a301309c7f4fd5fcbd7c1d4
idPersonaje
"P_NINFA"
nombre
"Estigia, la Ninfa"
descripcion
"Representante de la raza feérica. Diplomática que viste con ropas que ..."
```

4. Justificación del uso de MongoDB en el proyecto.

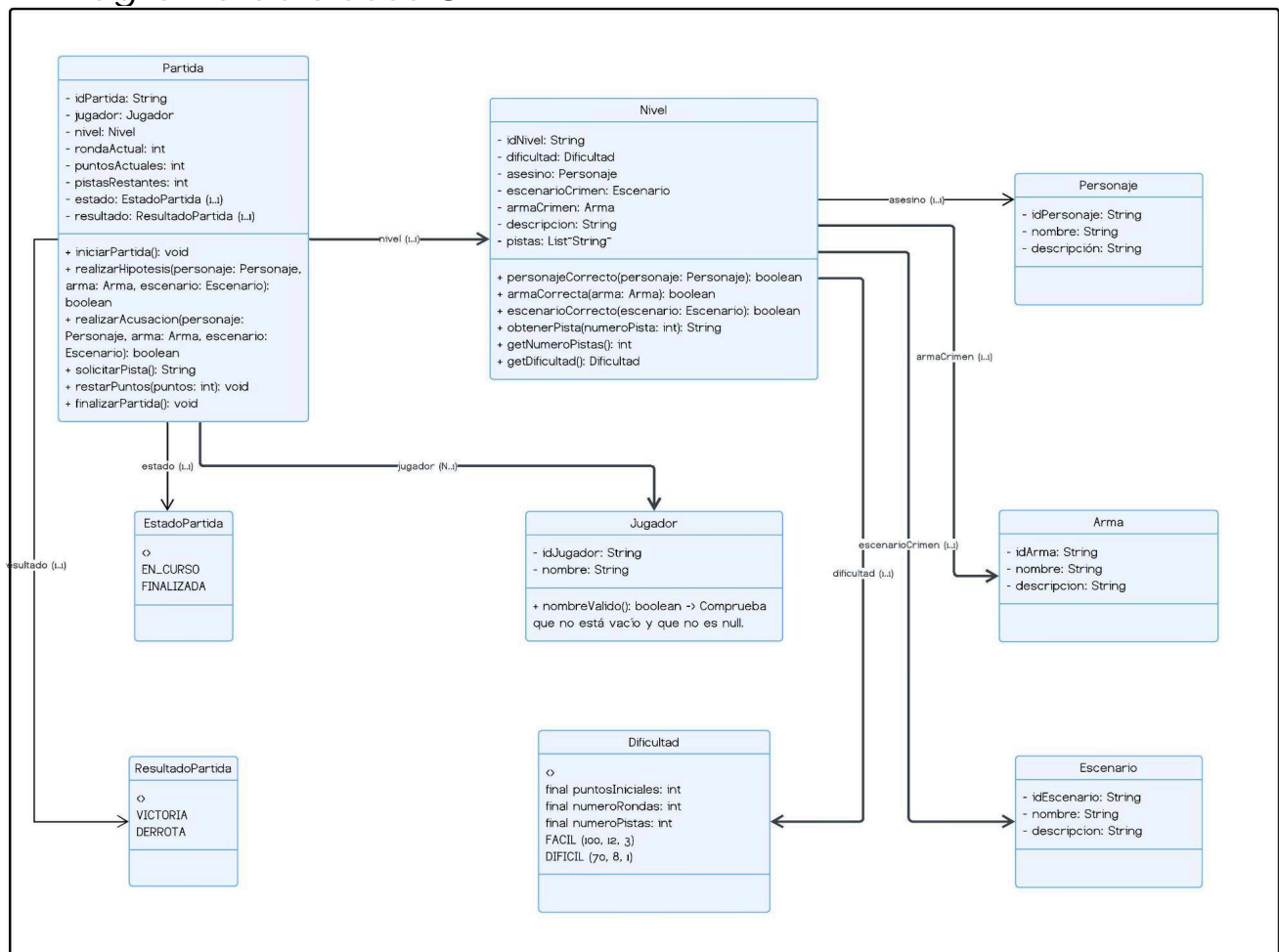
El diseño de nuestras clases está altamente jerarquizado (una Partida contiene un Nivel, que a su vez contiene un Personaje, un Escenario y un Arma). Si utilizáramos una base de datos relacional, necesitaríamos al menos 6 tablas distintas conectadas por claves foráneas, lo que complicaría las consultas SQL y el desarrollo de los DAOs. Al elegir MongoDB, podemos guardar el "árbol" completo de la partida en un solo documento JSON. Esto agiliza las operaciones CRUD desde Java, reduce los tiempos de desarrollo de nuestro patrón MVC y nos acerca al uso de estándares actuales de persistencia de datos.

5. Diseño inicial de la aplicación. Diagrama MVC

Capa Modelo

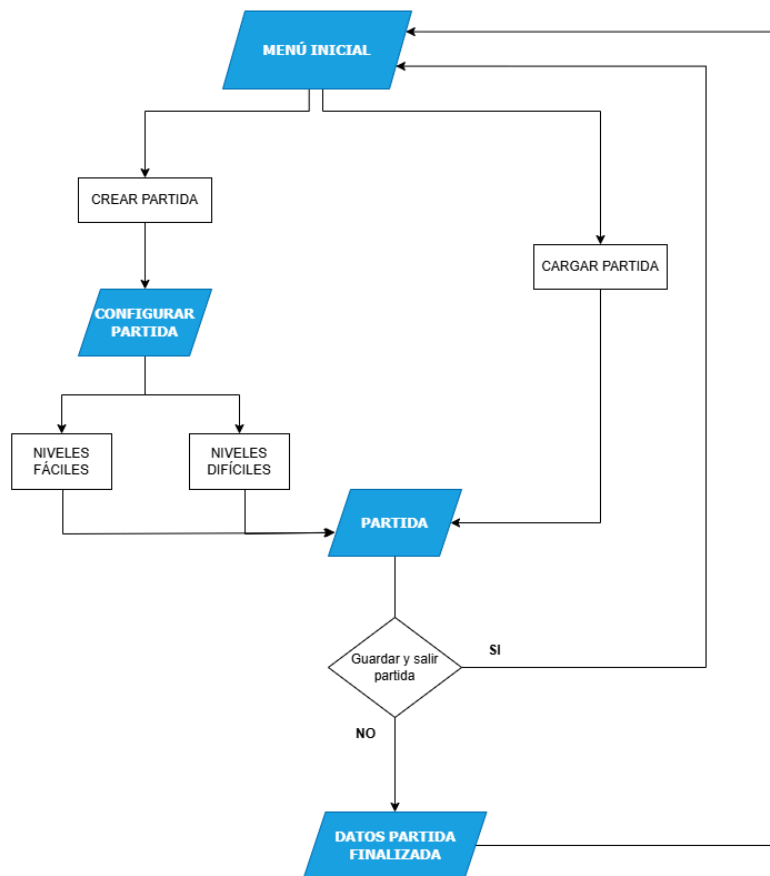
A la hora de implementar el programa, fue necesario ampliar clases auxiliares que facilitaran el trabajo con los controladores, especialmente con las funcionalidades de la partida. Se creó una clase Juego, que se usa como puente entre PartidaController y Partida. Además se crearon dos paquetes extra en modelo: factory y lore, con información más accesible para GUI, facilitando y aligerando el código de los controladores. El flujo principal se mantiene prácticamente igual.

Diagrama de clases UML



Capa Vista

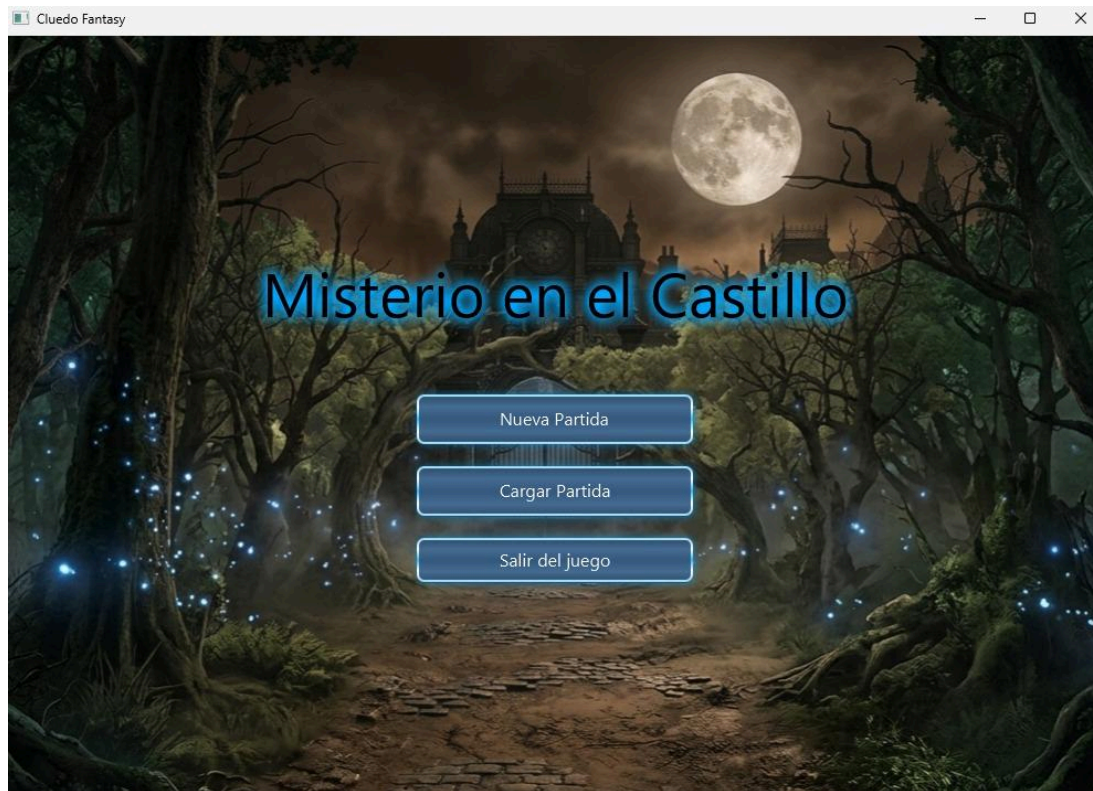
Diagrama de flujo



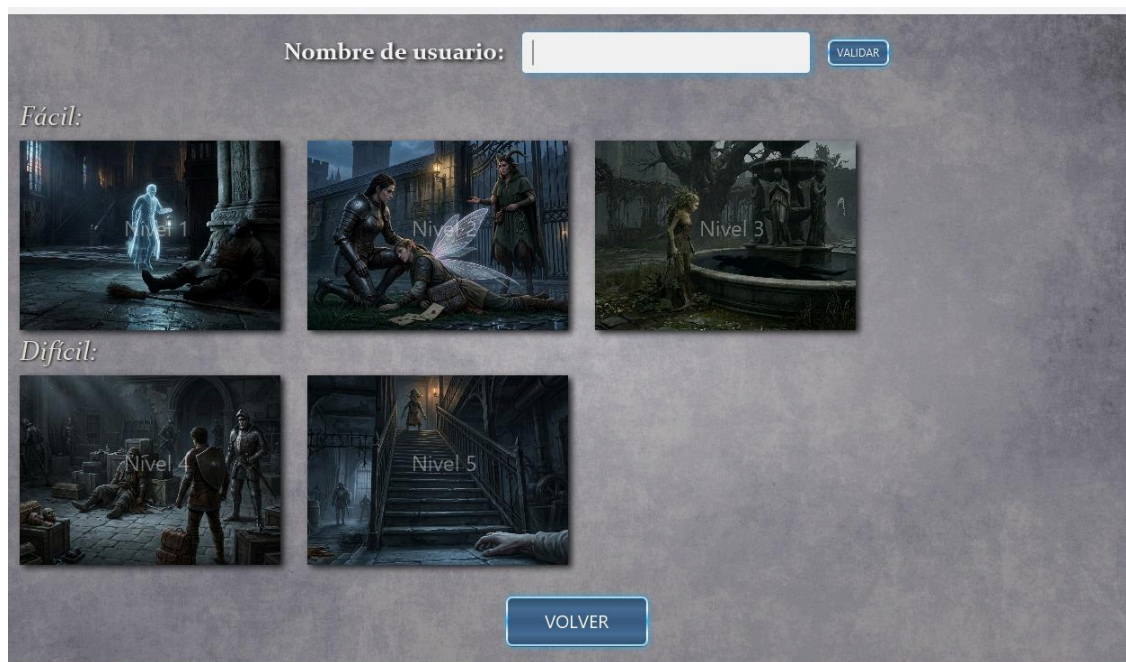
Trabajo Final: Misterio en el Castillo

Vistas

Vista 1: Menú de Inicio

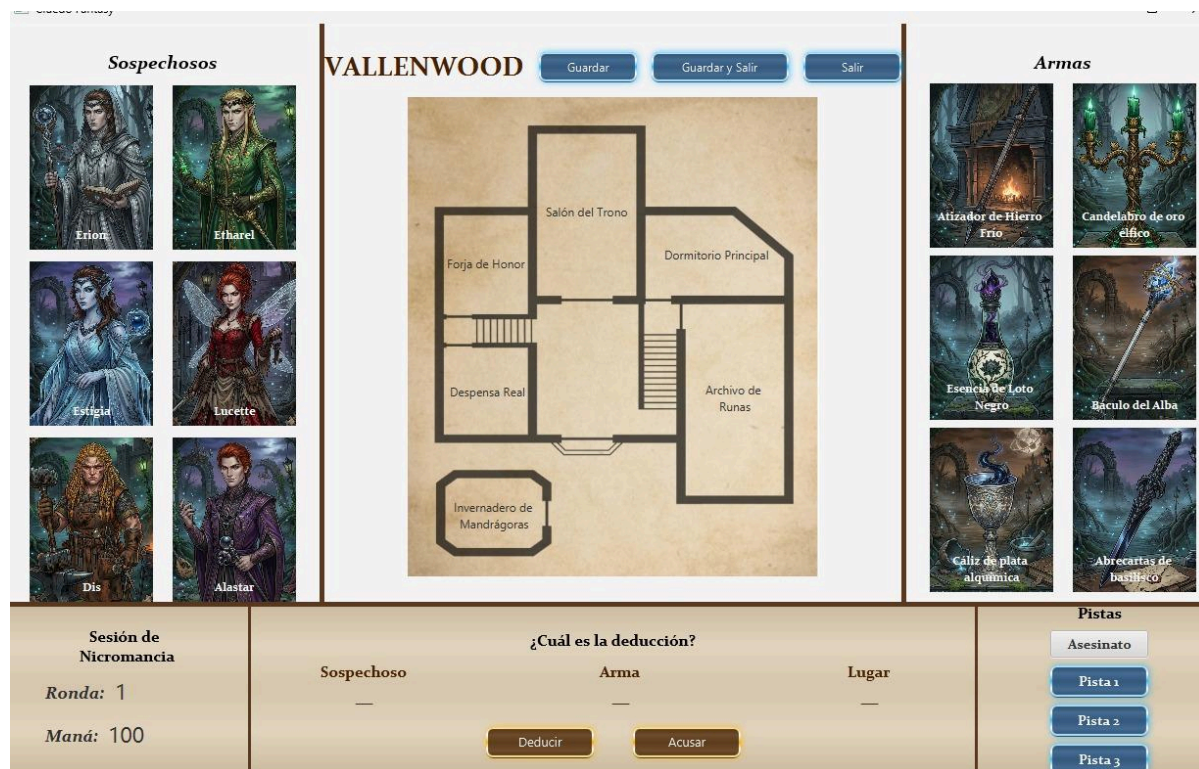


Vista 2: Menú de Partida

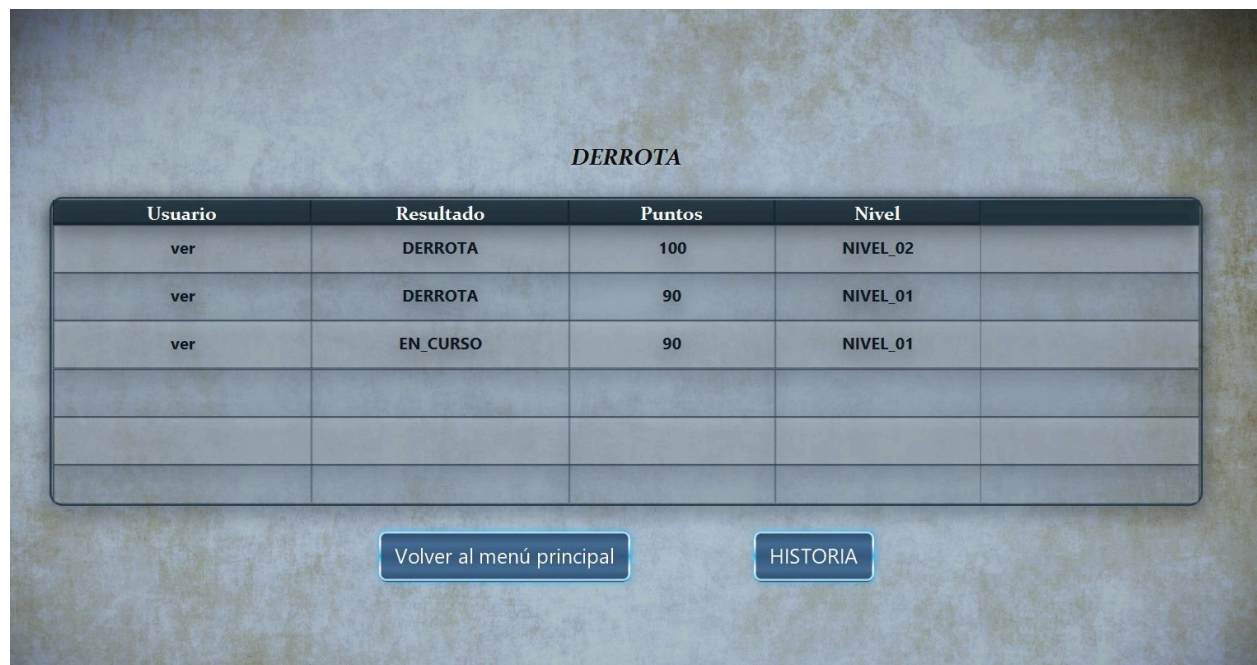


Trabajo Final: Misterio en el Castillo

Vista 3: Partida



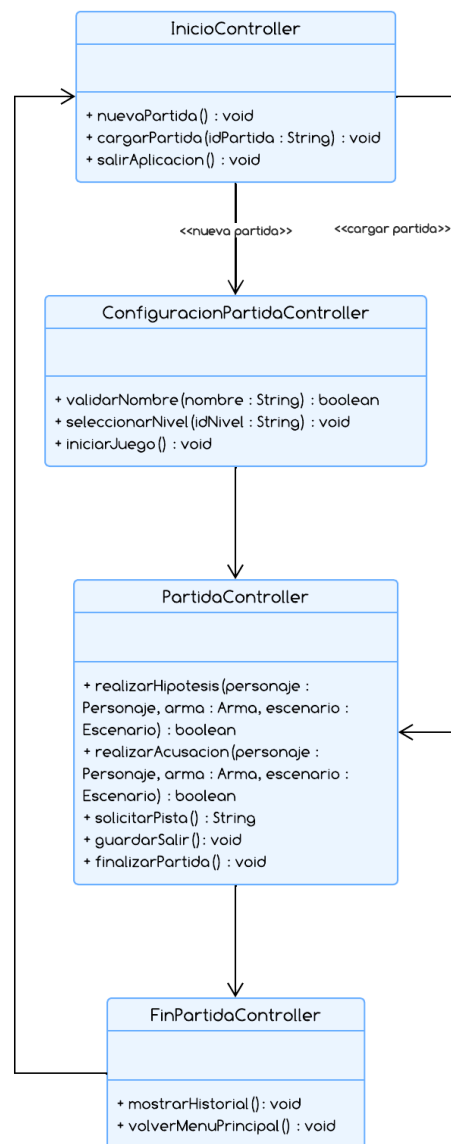
Vista 4: Final de Partida



Capa Controlador

Al igual que en capas anteriores, a la hora de poner en marcha el proyecto fue necesario hacer ajustes. En este caso, el número de métodos en PartidoController es sustancialmente mayor, para poder reflejar todas las funcionalidades, incluidas las opcionales.

Diagrama de Clases Controller



Capa Persistencia

Diagrama de Clases DAO

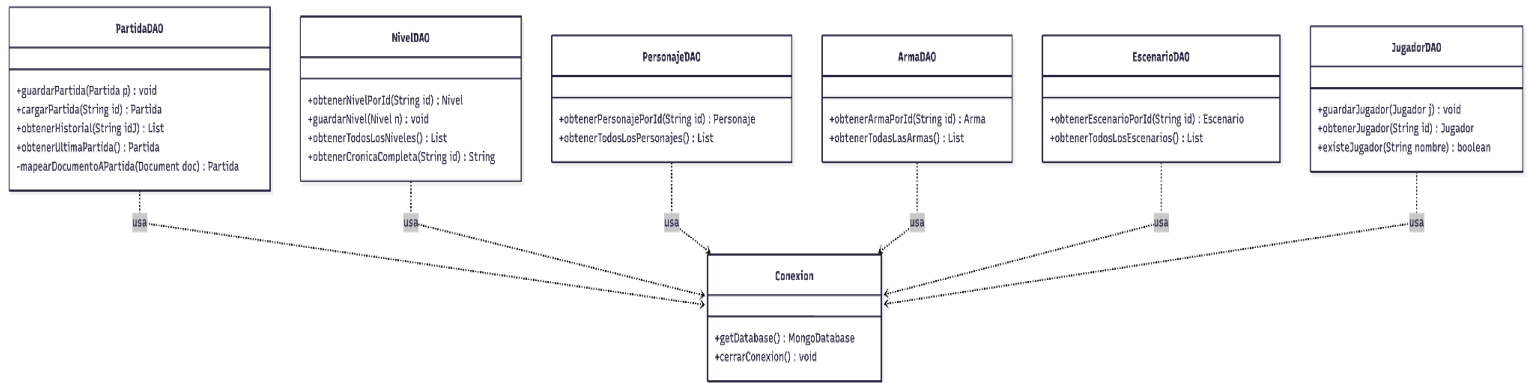


Diagrama Completo MVC

Patrón M-V-C

Diagrama de Clases M-V-C

A pesar de los cambios antes descritos, el flujo principal del programa, así como la arquitectura principal, sigue manteniendo la estructura básica, además de las clases y pequeñas reestructuraciones comentadas.

